

# Programspråk

Kapitel 11: ADT och OOP

Henrik Björklund



UMEÅ UNIVERSITY

Grundläggande insikt: abstraktion är bra (eller till och med nödvändig)

- När vi manipulerar data är det dess egenskaper vi är intresserade av, inte specifika detaljer.
- Lättare att hantera kod
- Vi människor fungerar så - vi behöver abstraktion
- Abstraktion låter oss diskutera saker på olika nivåer
- Detaljer är ibland ointressanta
- Förmåga att se ett system på olika nivåer

# Formell definition

ADT stack is elem + boolean + operations:

empty:  $\rightarrow$  stack

push: stack \* elem  $\rightarrow$  stack

pop: stack  $\rightarrow$  stack

top: stack  $\rightarrow$  elem

isEmpty: stack  $\rightarrow$  boolean

# Formell definition

ADT stack is elem + boolean + operations:

empty:  $\rightarrow$  stack

push: stack \* elem  $\rightarrow$  stack

pop: stack  $\rightarrow$  stack

top: stack  $\rightarrow$  elem

isEmpty: stack  $\rightarrow$  boolean

Equations:

isEmpty(empty) = true

isEmpty(push(s,e)) = false

pop(push(s,e)) = s

pop(empty) = error

top(push(s,e)) = e

top(empty) = error

# Program $\neq$ Algebra

Program innehåller detaljer om **implementationen**, inte bara abstrakt funktionalitet.

# Program $\neq$ Algebra

Program innehåller detaljer om **implementationen**, inte bara abstrakt funktionalitet.

ADT är ett försök att smälta ihop högnivåbeskrivningar från till exempel algebra med implementationsdetaljer.

Program innehåller detaljer om **implementationen**, inte bara abstrakt funktionalitet.

ADT är ett försök att smälta ihop högnivåbeskrivningar från till exempel algebra med implementationsdetaljer.

- Beskriv data och operationer på en högre abstraktionsnivå
- Isolera olika delar så att de inte kan användas av andra delar av programmet.
- Bara den utåt beskrivna funktionaliteten kan användas.

**Encapsulation:** En ADT specificeras som en syntaktisk enhet. Andra delar av programmet kan bara använda sig av definitionen.

# Encapsulation and Information Hiding

**Encapsulation:** En ADT specificeras som en syntaktisk enhet. Andra delar av programmet kan bara använda sig av definitionen.

**Information hiding:** Den interna representationen är inte tillgänglig utanför enheten.

**Encapsulation:** Bättre organisation, lättare att läsa, lättare att modifiera, separatkompilering.

**Encapsulation:** Bättre organisation, lättare att läsa, lättare att modifiera, separatkompilering.

**Information hiding:** Pålitlighet – ingen som inte har “behörighet” kan ändra på informationen.

## Abstraktion på olika nivåer

- Primitiva datatyper
- Procedurer/funktioner
- Moduler
- Records
- ADT
- Hierarkiska moduler/paket
- Objektorienterad programmering

- Data i centrum, inte kod
- Inkapsling
- Arv
- Dynamisk bindning
- Modellering

- 1963 definition av objekt/instanser
- 1967 Simula
- 1972 Smalltalk
- 80-talet: Java, C++

- Objekt som skickar och svarar på meddelanden
- Klasser, instanser och “medlemmar”
- Klass-metoder och instans-metoder
- Går att jämföra med trådar

# Support som byggts in i existerande

- C++, Objective-C (nya namn och annorlunda syntax, men i grunden utbyggnader av C)
- Ada 95
- CLOS
- Scheme

- Eiffel
- Java
- C#

- Smalltalk
- Ruby
- Squeak (smalltalk)

- Fält kan vara öppna eller stängda
- Återanvändning - kan vara besvärligt med ADT
- Ibland vill man låta viss kod se implementationsdetaljer

- private, **protected**, public
- Flexiblare
- Bryter mot definitionen av ADT
- Omdefinition av metoder (kan ge upphov till “shadowing”)
- En variabel kan ta olika typer
- Dynamisk bindning – en variant av polymorfism

- Bara objekt: Enkelt, elegant, långsamt
- En blandning: Snabbare, specialfall

# Dynamisk bindning vs statisk

- Flexibilitet
- Hastighet
- Tillåt båda?

- Dynamisk kontroll
  - Kostar tid och resurser
- Krav på samma parametrar/returvärde
  - Statisk kontroll
  - Mindre kostsamt

# Hur ser ni på arv?

- Fördelar?
- Nackdelar?
- Enkelt eller multipelt?

# Var skapas objekten?

- Stack
- Heap
- Fördelar/nackdelar

- Har setts på AP-javan
- Fördelar?
- Nackdelar?
- Användningsområden?

# Setters och getters

```
class Demo:
    def __init__(self,value):
        self.myValue = value
    def value(self):
        return self.myValue
    def setValue(self,newValue):
        self.myValue = newValue

d = Demo(12)
print(d.value())
d.setValue(17)
print(d.value())
```

# Setters och getters

```
class Demo:
    def __init__(self,value):
        self.myValue = value

    @property
    def value(self):
        return self.myValue
    @value.setter
    def setValue(self,newValue):
        self.myValue = newValue

d = Demo(12)
print(d.value())
d.setValue(17)
print(d.value())
```

- Ren OOP
- Allt är objekt
- Full programmeringsomgivning med GUI
- Allt är dynamiskt
- Enkelt arv
- Vill du testa? Ladda ner squeak

```
fac | result |  
  result <- i <- 1.  
  [i < self]  
  whileTrue: [i <- i+1.  
    result <- result * i].  
  ^result.
```

- Mycket vanligt
- Blandat typsystem
- Stort och (i vissas tycke onödigt) komplext
- Accesskontroll
- Multipelt arv

- Mycket vanligt
- Blandat typsystem
- Stort och (i vissas tycke onödigt) komplext
- Accesskontroll
- Multipelt arv
- Läs intervjuerna med Bjarne Stroustrup!

- Allt utom primitiva typer är objekt
- Wrapper classes
- Heap-dynamic
- Enkelt arv, men har interfaces

# Vad är “concurrency”?

- Enkelt uttryckt: flera saker händer samtidigt
- Olika typer:
  - Samma sak på många dataobjekt
  - Olika saker på olika dataobjekt
- Olika varianter
  - “Sant jämnloppande” – flera fysiska beräkningsenheter
  - “Logiskt jämnloppande” – simulering av flera fysiska beräkningsenheter

- Maskininstruktioner
- Uttryck
- Funktioner/procedurer/metoder
- Program

# Vad vi intresserar oss för här

- HPC – High Performance Computing: Pressa varje droppe prestanda ur systemet. Vetenskapliga beräkningar. Varje klockcykel är av intresse. Inte vårt fokus här
- Många olika saker ska göras samtidigt på olika ställen. Mer intressant för oss.

# Varför concurrency?

- Verkligheten är parallell (kognition, organisation, abstraktion)
- Bättre respons från systemet
  - Exempel: Ett användargränssnitt använder trådar för att få bättre responstid från systemet (inte hastighet i exekvering utan i reaktionstid).
- Distribution av program över flera kärnor/processorer/maskiner

# Vad är problemet?

Princip: Vi vill att systemet ska kunna utföra flera olika saker samtidigt.

# Vad är problemet?

Princip: Vi vill att systemet ska kunna utföra flera olika saker samtidigt.

För att det ska vara någon skillnad mot att köra flera program separat från varandra måste de kunna kommunicera med varandra.

# Vad är problemet?

Princip: Vi vill att systemet ska kunna utföra flera olika saker samtidigt.

För att det ska vara någon skillnad mot att köra flera program separat från varandra måste de kunna kommunicera med varandra.

Målet är att utbyta information och samordna användandet av resurser.

- För att kunna kommunicera är det nödvändigt att se till att en eller flera trådar/processer inte ändra data som andra trådar/processer just använder.
- Trådar/processer måste kunna synkronisera användandet av resurser.